

The Evidence Closure Loop

Engineering a Bounded-Autonomous, Multi-Agent Security Staff for Evidence-Grounded Assessments and Bug Bounty

A practitioner architecture and case study

Irlan de Alvarenga Cidade

Version 1.0 | July 10, 2026

Based on the public miniCISO v0.5.0 / Service Catalog v5 state

Licensed under Creative Commons Attribution 4.0 International (CC BY 4.0) | DOI: 10.5281/zenodo.21731494

Executive Summary

Security teams are increasingly experimenting with large-language-model agents, yet most implementations optimize for generation speed rather than decision integrity. A system that can produce a polished report faster can also produce a polished mistake faster. This practitioner whitepaper presents an architecture for a scalable security staff built around role separation, explicit evidence models, mandatory adversarial quality assurance, and bounded autonomy.

The implementation uses a central chief-of-staff, specialized security SMEs, a shared evidence workspace, and an independent Security QA gate. It applies Knowledge Augmented Generation (KAG) in two distinct planes. The retrieval plane converts a security hypothesis into a structural query, a minimal provenance-preserving evidence pack, and controlled expansion to the raw artifact. The decision plane links program scope, vulnerability semantics, evidence strength, trust boundaries, and qualification criteria before allowing a finding to proceed. External candidate findings receive one of three decisions: GO, RESEARCH, or NO-GO.

The architecture also includes a governed self-update capability. The live deployment may prepare sanitized public changes through an allowlisted export, secret and operational-state scanning, human diff review, tests, and a pull request. This is not unrestricted self-modification; it is controlled promotion of reproducible state.

The paper contributes a transferable operating model, a dual-plane KAG pattern, an evidence-closure discipline, a triage-first bug bounty gate, and an evaluation framework that measures decision quality in addition to cost. The MiniCISO implementation is used as a reproducible case study, not as a product pitch.

Central thesis

Useful security autonomy is not the ability to act without humans. It is the ability to produce a defensible decision under explicit evidence, scope, and authority constraints.

Key contributions

- A role-separated staff model with a chief-of-staff, specialized security SMEs, and mandatory independent QA.
- An evidence-closure discipline that distinguishes declared configuration, effective runtime, and validated behavior.
- A dual-plane KAG pattern: hypothesis-guided selective retrieval and graph-structured decision validation.
- A bug bounty gate that returns GO, RESEARCH, or NO-GO before any external report drafting.
- A governed self-update workflow that exports only sanitized, allowlisted, reviewable state through tests and pull requests.

Scope note

The MiniCISO repository and service catalog are used as a concrete, reproducible implementation. The paper does not argue that this repository is a commercial product, nor that its current telemetry proves general superiority over other agent frameworks.

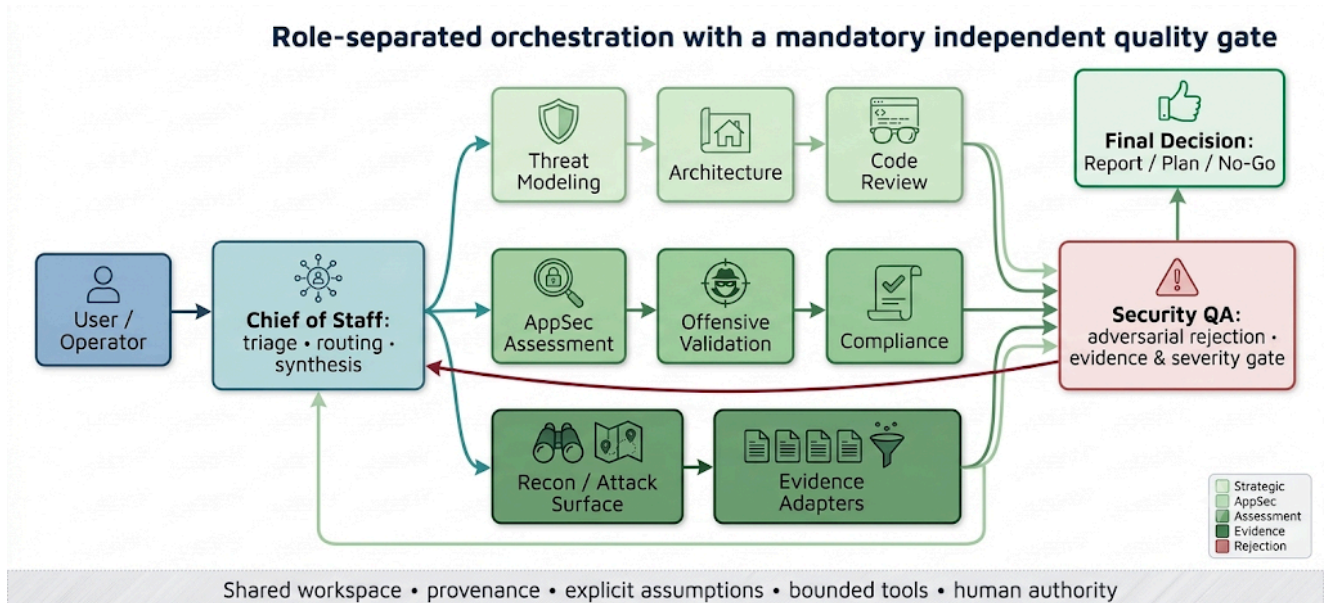


Figure 1 — Role-separated orchestration. Final outputs must pass through an independent Security QA gate.

1. Introduction: the bottleneck is not generation

Security assessment is a decision problem disguised as a reporting problem. The visible output is usually a report, a threat model, a finding, or a remediation plan. The difficult work happens earlier: choosing the correct lens, identifying the evidence that would change the decision, distinguishing a product feature from a boundary violation, deciding when uncertainty is acceptable, and knowing when not to escalate a technical observation into a vulnerability claim.

A single general-purpose agent can imitate many of these activities. It can summarize scanner results, explain a code path, write a vulnerability narrative, and suggest fixes. The weakness is that all of those functions occur inside the same conversational role. The same agent that discovers an exciting primitive may also become invested in proving it is a finding. The same context window contains raw evidence, hypotheses, previous conclusions, and persuasive prose. This creates predictable failure modes: confirmation bias, evidence laundering, severity inflation, scope drift, and reports that sound more certain than the underlying evidence.

The central design claim of this work is that useful autonomy does not come primarily from a more capable model. It comes from a system architecture that separates responsibilities, preserves provenance, forces explicit state transitions, and grants independent components the authority to reject work. In this model, the most valuable agent is often not the one that finds something. It is the one that says the evidence is insufficient.

2. Problem statement and design goals

The target environment is a small security function that must handle heterogeneous work: architecture review, code review, AppSec assessment, threat modeling, authorized offensive validation, compliance mapping, attack-surface triage, and bug bounty research. The volume and variety of evidence can be high, while expert attention remains scarce.

The design goals are therefore broader than “generate a good answer.” The system must route work to a suitable specialist, preserve the source and strength of each material claim, distinguish declared configuration from effective state and observed behavior, control offensive actions through authorization and scope, prevent weak bug bounty candidates from becoming polished submissions, reduce context cost without inventing absence, and evolve without copying private state into a public repository.

Five non-goals are equally important. The system is not an autonomous penetration-testing platform, not an unsupervised remediation engine, not a replacement for accountable security leadership, not a guarantee against hallucination, and not a claim that every specialized role requires a separate model. The architecture treats agents as controlled roles with contracts, not as independent personalities.

3. Contributions

This case study makes five practical contributions.

First, it defines a role-separated operating model in which a chief-of-staff performs intake, decomposition, routing, and final synthesis; specialized SMEs perform domain analysis; and Security QA independently approves, approves with caveats, or rejects the output.

Second, it introduces an evidence-closure model with three layers: declared configuration, effective runtime, and validated behavior. Conclusions must be calibrated to the strongest layer actually observed, and disagreement between layers becomes a first-class “Configuration Divergence” finding rather than being silently collapsed.

Third, it applies KAG in two complementary ways. Hypothesis-guided selective retrieval reduces irrelevant context while preserving path, ancestry, inclusion rationale, and source hash. A separate decision graph connects scope rules, vulnerability semantics, evidence, exclusions, trust boundaries, and submission status.

Fourth, it formalizes an adversarial pre-submission gate for external findings. A technical primitive is not allowed to become a report merely because it resembles a familiar vulnerability class. The gate must prove qualifying impact or block submission.

Fifth, it defines governed evolution through a sanitized self-update pipeline: allowlisted export, pattern scanning, human diff review, validation, tests, branch, and pull request.

4. Architecture overview

The public implementation is a reproducible overlay on top of Hermes Agent rather than a fork of the runtime. The runtime version and commit are pinned, installer hashes are verified, profiles and prompts are recreated by bootstrap, and credentials remain local. This choice separates the operating method from the underlying agent runtime and makes rollback and review more tractable.

The current public staff contains nine profiles: one chief-of-staff, seven domain specialists, and one Security QA profile. The specialists cover threat modeling, architecture, code review, AppSec assessment, offensive validation, compliance mapping, and recon/attack-surface strategy. The shared workspace provides stable locations for inputs, drafts, QA artifacts, reports, and templates.

The coordinator is intentionally not the final authority on quality. It may route and synthesize, but a final report must pass through Security QA. This creates an organizational control analogous to separation of duties: the component rewarded for completing the work is not the only component deciding whether the work is acceptable.

Role	Primary responsibility	Hard boundary
Chief of Staff	Intake, decomposition, routing, synthesis	Final authority over quality; bypass QA
Threat Modeling	Assets, actors, trust boundaries, abuse cases	Treat assumptions as verified architecture
Security Architecture	IAM, secrets, segmentation, resilience, runtime controls	Conclude from declared config alone
Code Review	File/line evidence, exploit conditions, remediation, tests	Generalize beyond inspected code
AppSec Assessment	Authn/authz, API/web risk, scanner validation, supply chain	Promote scanner output without validation
Offensive Validation	Safe, authorized tests and exploitability analysis	Operate outside explicit scope or stop conditions
Compliance Mapping	Map evidence and findings to frameworks	Turn control mapping into proof of effectiveness
Recon / Attack Surface	Low-noise mapping and hypothesis prioritization	Call recon output a final finding
Security QA	Reject unsupported scope, evidence, severity, and conclusions	Become a stylistic editor only

5. Role contracts and handoffs

Specialization only helps when role boundaries are explicit. Otherwise, multiple agents become redundant paraphrasers. Each role therefore has a mission, scope, evidence expectations, safety boundaries, and a defined handoff.

The chief-of-staff receives the objective, scope, context, artifacts, constraints, and desired output. It decomposes the request and chooses the smallest set of SMEs needed. The selected SME produces a technical draft and records assumptions, confidence, residual risk, and next steps. Security QA then evaluates scope, evidence quality, severity, runtime validation, missing context, and format. The chief-of-staff incorporates the QA decision into the final response.

Recon is deliberately separated from finding validation. Recon maps surfaces and produces hypotheses; it does not produce final vulnerabilities. Offensive validation is also bounded: external work requires explicit authorization, target scope, limits, and stop conditions. This prevents a workflow optimization from becoming an authorization bypass.

The handoff artifact matters as much as the agent. A reliable handoff contains the question being answered, the evidence examined, evidence not examined, claims with support level, unresolved alternatives, and the next decision. This makes the workflow inspectable even when different models or tools are substituted later.

6. Evidence closure: declared, effective, validated

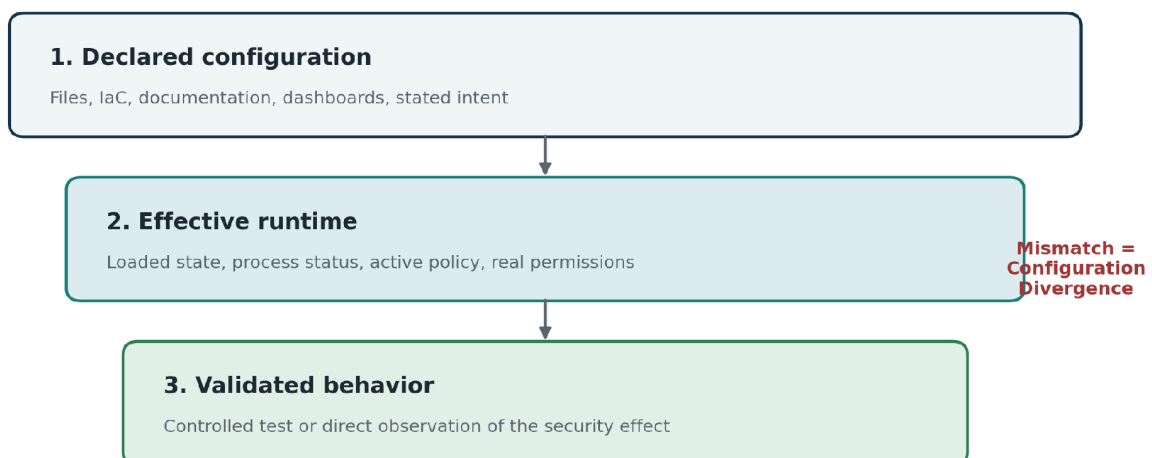
Security conclusions frequently become unreliable because evidence from different layers is treated as equivalent. A configuration file can show intent. It does not prove that the runtime loaded the file, that another policy did not override it, or that the system behaves as expected.

The architecture therefore separates three layers. Declared configuration includes files, infrastructure-as-code, dashboards, documentation, and stated policy. Effective runtime includes the actual loaded configuration, active process state, applied firewall rules, effective permissions, running containers, or gateway status. Validated behavior is a controlled test or direct observation of the security effect.

When the layers disagree, runtime and observed behavior take precedence for risk assessment. The disagreement is itself useful evidence and is reported as Configuration Divergence. When runtime or behavior has not been validated, the conclusion uses language such as “configured to,” “declared only,” or “not validated,” and confidence is reduced.

This discipline changes the behavior of the QA agent. It can reject phrases such as “the service is not exposed” when the only support is a configuration file. It can require effective-state commands or a safe behavior test. The result is not only better wording; it is a different epistemic contract for the system.

Evidence Closure Model



A conclusion cannot be stronger than the strongest layer actually validated.

Figure 2 — Evidence closure. A file can prove intent; it cannot by itself prove effective state or observed behavior.

Language rule

When runtime or behavior is missing, say “configured to,” “declared only,” or “not validated.” Do not say “is protected” or “is not exposed.”

7. Dual-plane KAG

The term KAG is used carefully in this work. Academic Knowledge Augmented Generation combines structured knowledge, graph relationships, retrieval, and reasoning to improve professional-domain answers. The current implementation is KAG-oriented rather than a complete reproduction of the academic framework. It adopts the central principle that security reasoning should operate over explicit entities, relationships, rules, and evidence rather than only similarity-ranked text chunks.

The architecture has two planes.

The retrieval plane begins with a hypothesis, not with the entire artifact. A structural index records JSON path, parent path, ancestors, estimated tokens, signals, a content preview, and a source hash. A KAG query states the security surface, question, target claim, impact target, anti-goals, required evidence types, token budget, confidence threshold, and raw-expansion policy. A deterministic selector scores nodes and emits a minimal retrieval pack with inclusion reasons and selection metrics.

The operating contract is strict: absence from the retrieval pack is never evidence of absence from the raw artifact. If required evidence is not present, the correct state is `not_verified_in_raw`. Final claims remain raw-authoritative, and the system may expand to the raw source when the decision depends on missing evidence.

The decision plane models the relationships needed to approve a security conclusion. For external findings, it uses four linked knowledge sets: a Scope Graph, Vulnerability Graph, Evidence Graph, and Decision Graph. These structures connect program rules, expected product behavior, trust boundaries, candidate abuse paths, claim-to-evidence links, exclusions, missing criteria, and final state.

The two planes solve different problems. The retrieval plane controls what enters context. The decision plane controls what may be concluded. Conflating them would create a dangerous shortcut: a highly relevant slice can still be insufficient evidence, and a complete evidence set can still describe expected behavior rather than a vulnerability.

Dual-plane KAG: retrieving less, deciding better

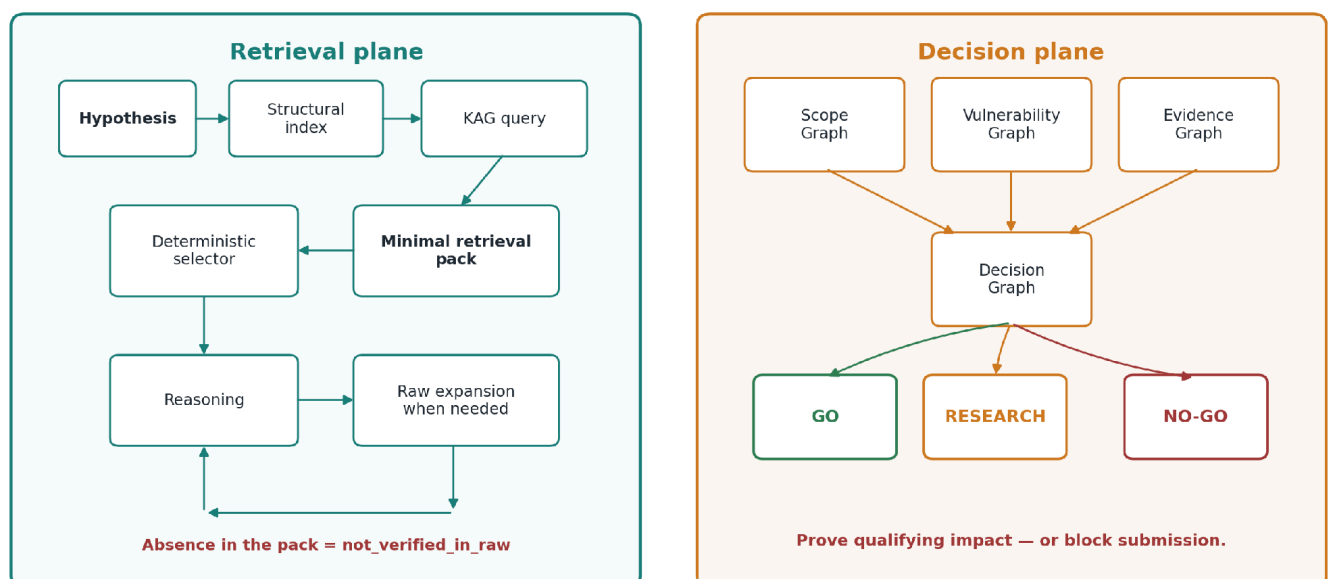


Figure 3 — KAG is applied separately to context selection and decision qualification.

8. Headroom: from compression-first to selection-first

The Headroom work began as a response to a practical cost problem: large structured artifacts could dominate the model context. Compression improved the ratio, but it still spent resources reading and transforming information that was irrelevant to the current hypothesis.

The next step changed the order of operations from “entire artifact → compression → reasoning” to “hypothesis → structural selection → minimal retrieval pack → optional compression → reasoning → raw expansion when necessary.” This is more than a cost optimization. It also forces the system to state what it is looking for, what evidence types would support the claim, and what should be excluded as noise.

Operator telemetry from the earlier compression phase recorded 1,169,417 tokens saved across six runs in which savings were positive, or approximately 194,903 tokens per effective run. Including three no-op runs, the average was approximately 129,935 tokens across nine enabled runs. A deduplicated latest-run view across five unique artifacts recorded 552,208 tokens saved, or approximately 110,442 per artifact. These numbers are implementation telemetry, not a controlled benchmark, and they apply to the compression phase rather than proving the quality of selection-first retrieval.

For the selection-first phase, token savings are insufficient as a success criterion. The public design tracks source and selected token estimates, selection ratio, raw expansion rate, retrieval precision and recall, false-omission events, and decision delta. Decision delta is particularly important: if the reduced context changes a security decision compared with raw review, the optimization may be unsafe even when it is cheaper.

Metric	Observed value	Interpretation
Compression phase — effective runs	6	Runs with tokens_saved > 0
Compression phase — total saved	1,169,417 tokens	Observed operator telemetry
Mean per effective run	~194,903 tokens	Not a controlled benchmark
All Headroom-enabled runs	9	Includes three no-op executions
Mean across all enabled runs	~129,935 tokens	Conservative view
Latest run per unique artifact	5 artifacts / 552,208 tokens	Deduplicated operational view
Mean per unique artifact	~110,442 tokens	Compression results, not selection recall

9. Bug bounty and external finding governance

Bug bounty work exposes a distinctive failure mode for agentic systems: a technically interesting primitive creates narrative momentum. Once a system recognizes a pattern such as SSRF, IDOR, unsafe deserialization, or token exposure, it can rapidly produce a persuasive report that assumes impact rather than proving it.

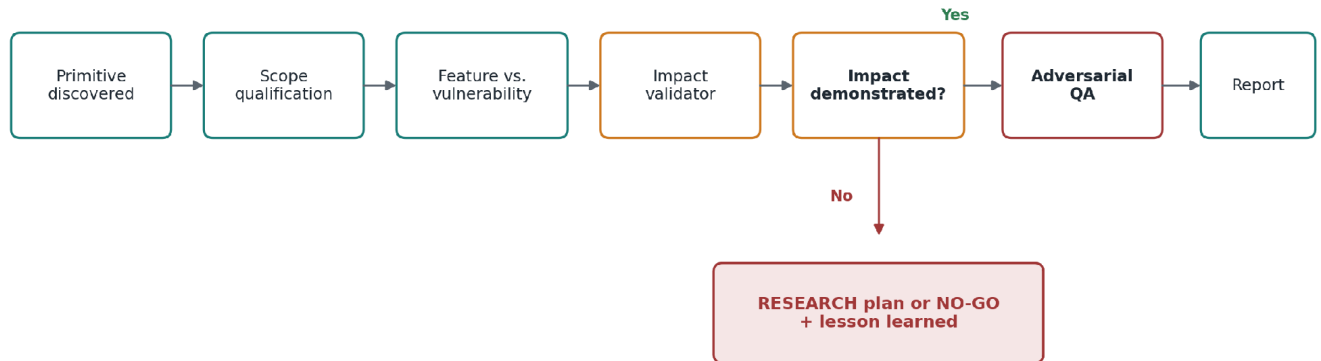
The architecture reverses the workflow. It does not begin by asking how to turn the evidence into a good report. It asks whether the evidence would survive triage as a qualifying finding for this specific program.

The mandatory gate has five stages. Scope Qualification checks assets, exclusions, impact requirements, and authorization. Feature versus Vulnerability distinguishes the intended product function from an abusive boundary crossing. Impact Closure requires a concrete sentence: because of this behavior, an attacker can perform a specific unauthorized action, causing an observable consequence that normal authorized functionality would not permit. Evidence Sufficiency labels each material claim as DIRECT, SUPPORTED_INFERENCE, SPECULATIVE, or UNSUPPORTED. Adversarial QA then attempts to reject the finding because of expected behavior, exclusion, missing trust-boundary break, unrealistic preconditions, weak evidence, incorrect classification, or inflated severity.

Only three decisions are allowed. GO permits report drafting. RESEARCH produces a safe, authorized impact-validation plan. NO-GO blocks submission and records the lesson learned. If demonstrated impact is false, GO is impossible.

This design is intentionally conservative. A false negative can be revisited when new evidence appears. A polished false positive consumes triage capacity, damages researcher credibility, and teaches the system the wrong abstraction.

External finding decision flow



A technical primitive is an input to validation, not a vulnerability by definition.

Figure 4 — Report generation begins only after impact is demonstrated and adversarial QA fails to reject the finding.

Gate	Question	Reject when...
Scope	Asset/class in scope; exclusions; authorization	Unsupported external testing or excluded class
Boundary	Expected function vs. security boundary	No additional unauthorized capability
Impact	Concrete action and observable consequence	"May allow," "could lead to," theoretical impact
Evidence	DIRECT or supported inference for material claims	Speculative/unsupported claim drives title or severity
Severity	Preconditions, privilege, reach, consequence	Severity exceeds demonstrated effect
Alternatives	Legitimate explanation considered	Single interpretation presented as fact

10. Sanitized example: a technically correct SSRF primitive that should not be submitted

Consider a URL import feature. The endpoint accepts an external URL, the backend performs a request, a callback is observed, and redirects are followed. These observations prove that the import mechanism works and establish properties of the request primitive.

They do not automatically prove a security boundary violation. If the program excludes blind SSRF without direct impact, the callback alone does not qualify. If no internal service, sensitive data, privileged action, or trust-derived effect is demonstrated, the required impact remains absent.

The Evidence Graph can support "the backend performed a request to an attacker-controlled destination" as DIRECT. It cannot support "the attacker accessed the internal network" without evidence unique to an internal resource. The Decision Graph therefore records the exclusion, required impact, demonstrated impact, prohibited inferences, and strongest rejection argument.

The correct result is RESEARCH when a safe and authorized path to impact validation remains, or NO-GO when it does not. Repeating the callback with more headers, source IPs, user agents, or redirect chains may strengthen proof of the same primitive but does not close the impact gap. The next experiment must test a security consequence, not merely re-demonstrate behavior.

11. Security QA as an adversarial institution

Quality assurance is often implemented as a final editing pass. That is too late and too weak for autonomous security analysis. The QA role in this architecture is adversarial toward the conclusion itself.

Its first job is to reject the work if possible. It checks whether the analysis stayed within authorized scope, whether the claimed security property was actually tested, whether severity matches evidence, whether an alternative legitimate explanation exists, whether a scanner result was mistaken for a confirmed issue, and whether assumptions are visible. Only after it fails to reject the conclusion does it improve clarity and presentation.

This creates a useful asymmetry. Discovery components may be optimized for recall and hypothesis generation. QA may be optimized for precision, evidence quality, and decision defensibility. The chief-of-staff balances the two rather than asking one model to maximize both at the same time.

The QA decision is explicit: Approved, Approved with caveats, or Rejected. A rejected output returns mandatory corrections or evidence requests. A final deliverable must include an executive summary, findings, recommendations, assumptions, confidence level, residual risk, and next steps. The format is not bureaucracy for its own sake; it prevents uncertainty and residual risk from disappearing during synthesis.

“The first job of Security QA is not to improve the wording. It is to reject the conclusion if the evidence cannot carry it.”

12. Governed autonomy and safe self-update

A mature agentic system must evolve, but “self-update” is a dangerous phrase. Unrestricted self-modification would combine write access, execution authority, and policy authority in one component. The implementation instead uses a governed promotion pipeline.

The live VPS may contain improved Headroom/KAG tools and non-secret configuration that should be reflected in the public overlay. An export script copies only an allowlisted set of files, sanitizes the public configuration, and scans text for suspicious token patterns, private keys, authorization headers, active-engagement markers, memories, sessions, cron output, and raw assessment traces. The result is reviewed through git diff, validated, tested, committed to a dedicated branch, and opened as a pull request.

The public repository explicitly excludes environment files, credentials, cookies, memories, sessions, logs, raw assessment artifacts, active program references, and sensitive findings. Human review remains mandatory before commit and merge. Rollback is available through previous repository commits and bootstrap.

The important distinction is between preparation authority and publication authority. The agent may prepare, sanitize, test, and propose a change. A human retains authority to approve the public state. This is bounded autonomy with an auditable change path, not an agent rewriting its own rules in production.

Governed evolution: safe self-update without self-rewrite



Autonomy prepares and validates change; authority to publish remains explicit.

Figure 5 — The agent may prepare and validate a change, while publication authority remains explicit and reviewable.

13. Failure-driven evolution

The architecture emerged from failures rather than from a clean-room design.

One class of failure was epistemic: configuration was treated as proof of runtime security. The response was the three-layer evidence model and QA language rules.

A second class was operational: a dependency watcher silently received HTTP 422 because it requested a page size of 100 while the upstream API accepted a maximum of 20. The process itself returned misleadingly little information. The lesson was “who watches the watcher”: health checks must validate the monitoring path, API constraints, and observable output rather than only the existence of a scheduled job.

A third class was economic: verbose intermediate reports and full-artifact processing consumed large contexts. Compression reduced cost, but selection-first KAG was required to stop sending irrelevant material in the first place.

A fourth class was methodological: bug bounty candidates could be overdeveloped around a technical primitive before program-specific impact was proven. The response was the GO / RESEARCH / NO-GO gate and an Impact Validator role between discovery and drafting.

A fifth class was governance: useful changes developed on the live system risked drifting away from the reproducible public overlay. The response was a sanitized export and pull-request workflow.

Each failure became a control, metric, or state transition. This is a more durable path to maturity than adding more personas.

14. Evidence sources and extensibility

The staff is designed to consume heterogeneous evidence rather than depend on a single scanner. Inputs may include source code, diffs, architecture diagrams, infrastructure configuration, runtime command output, logs, SAST/DAST/SCA findings, SBOMs, dependency graphs, container manifests, public repository metadata, and authorized external observations.

Sociotechnical evidence can also enrich supply-chain decisions. Contributor concentration, bus factor, cadence, turnover, and inactivity do not prove a vulnerability, but they can change prioritization and confidence. The same rule applies to every adapter: an external signal is evidence for a bounded claim, not a verdict.

This architecture supports adding specialists or tools without changing the central decision contract. A new SME must define what questions it answers, what evidence it requires, what it is prohibited from concluding, and how its output is reviewed. A new tool must preserve provenance and state whether it observed declaration, runtime, or behavior.

15. Evaluation framework

Agentic security systems should be evaluated on decision quality, not only response quality. A useful evaluation program contains four dimensions.

Decision integrity measures the percentage of material claims linked to evidence, the rate of QA rejection or correction, severity changes after QA, agreement between reduced-context and raw-authoritative review, and the number of prohibited inferences caught before delivery.

Retrieval quality measures precision, recall, false-omission events, raw expansion rate, and decision delta. Token savings without these measures can hide evidence loss.

Operational reliability measures bootstrap reproducibility, offline validation pass rate, profile smoke-test pass rate, update rollback success, stale-state detection, and monitor-path health.

Human value measures time to defensible decision, analyst review time, report rework, accepted versus rejected external submissions, and the quality of lessons learned from NO-GO decisions.

The current implementation provides operator telemetry and public tests but does not yet constitute a controlled comparative study. A stronger future evaluation would use a blinded corpus of security tasks, raw-authoritative

labels from experienced reviewers, multiple model configurations, and ablations for single-agent, multi-agent without QA, QA without KAG, and full dual-plane architecture.

16. Threat model for the staff itself

The agentic staff is also a security system and must be threat-modeled.

Prompt injection may arrive through source repositories, logs, web pages, issue text, or scanner output. The architecture should treat retrieved content as untrusted evidence, separate instructions from data, and constrain tool execution.

Tool abuse is possible when profiles have local terminal access. The public documentation therefore acknowledges that the local backend exposes the host and recommends stronger isolation such as containers or remote sandboxes when required by the threat model.

Data leakage can occur through prompts, logs, memories, update exports, or public commits. The self-update allowlist, secret scanning, sanitization, and exclusion rules reduce risk but do not eliminate the need for human review.

Cross-agent contamination can occur when unsupported claims are copied through handoffs and gain apparent legitimacy. Claim-to-evidence links, explicit confidence, QA rejection, and raw-authoritative review are the principal controls.

Policy drift can occur when prompts or runtime versions change. Pinned runtime commits, installer hashes, repository validation, smoke tests, and reviewable prompt files provide a change record.

17. Limitations

This work has important limitations.

The implementation is a practitioner case study operated by a single primary user, not an independently evaluated enterprise deployment. The reported token savings are observational and were collected before full selection-first evaluation. The public repository intentionally excludes real engagement artifacts, which limits external reproducibility of end-to-end results.

KAG-oriented structures do not guarantee correctness. A graph can encode a wrong rule, omit an important relationship, or give false confidence. Deterministic selection can still miss evidence. Raw expansion and human review remain necessary.

Role separation can create coordination overhead, duplicated context, and latency. A multi-agent system may perform worse than a strong single agent on simple tasks. The architecture should therefore route only when specialization or independent review adds value.

The QA agent is not independent in the organizational sense when it uses the same model family, prompts, or evidence as the analyst. Diversity of model, rubric, and reviewer can improve independence, but the process cannot claim full assurance.

Finally, bounded autonomy depends on tool permissions, deployment isolation, secrets management, and operator discipline. Prompt rules are not a substitute for operating-system controls.

Honest claim boundary

The architecture reduces specific failure modes; it does not guarantee correct findings, complete retrieval, or safe operation without system-level controls.

18. Transferable design principles

The following principles are intended to transfer beyond the MiniCISO implementation.

1. Optimize the decision pipeline, not the report generator.
2. Separate discovery, validation, rejection, and synthesis.
3. Make “not enough evidence” a successful system outcome.
4. Treat recon, scanner alerts, and technical primitives as inputs, not findings.

5. Distinguish declared configuration, effective runtime, and validated behavior.
6. Link every material claim to evidence and label the inference strength.
7. Retrieve by hypothesis and evidence need, not only semantic similarity.
8. Preserve provenance and make raw expansion an explicit fallback.
9. Measure decision delta, not only token savings.
10. Give QA authority to reject, not merely edit.
11. Keep offensive work authorization-aware and stop when the next test is out of scope.
12. Allow agents to prepare changes, but keep publication and policy authority reviewable.
13. Turn failures into controls, metrics, or state transitions.
14. Publish methodology and sanitized tooling without publishing engagement state.

19. Conclusion

A scalable agentic security staff is not created by multiplying personas. It is created by engineering a chain of accountable decisions.

The architecture described here uses a coordinator, specialized SMEs, an adversarial QA institution, a three-layer evidence model, dual-plane KAG, a triage-first external finding gate, and a governed self-update loop. Its most important behavior is the ability to stop: stop when runtime has not been validated, stop when a retrieval pack cannot prove absence, stop when a primitive lacks impact, stop when the next test is unauthorized, and stop before private state becomes public.

This is the practical definition of bounded autonomy for security work: the system can gather, select, reason, challenge, and propose, while its conclusions remain proportional to evidence and its actions remain constrained by authority.

References

- [1] I. A. Cidade, “miniCISO: agentic security staff,” public GitHub repository, 2026.
<https://github.com/icidade/miniCISO>
- [2] I. A. Cidade, “MiniCISO Staff Service Catalog v5,” July 10, 2026.
<https://github.com/icidade/miniCISO/blob/main/miniciso-staff-service-catalog-v5.md>
- [3] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” NeurIPS, 2020.
<https://arxiv.org/abs/2005.11401>
- [4] L. Liang et al., “KAG: Boosting LLMs in Professional Domains via Knowledge Augmented Generation,” 2024. <https://arxiv.org/abs/2409.13731>
- [5] D. Edge et al., “From Local to Global: A Graph RAG Approach to Query-Focused Summarization,” 2024.
<https://arxiv.org/abs/2404.16130>
- [6] Q. Wu et al., “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation,” 2023.
<https://arxiv.org/abs/2308.08155>
- [7] National Institute of Standards and Technology, “Artificial Intelligence Risk Management Framework (AI RMF 1.0),” NIST AI 100-1, 2023. <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf>
- [8] National Institute of Standards and Technology, “Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile,” NIST AI 600-1, 2024.
<https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf>

Source and evidence note

Repository details in this paper were checked against the public icidade/miniCISO repository and the supplied Service Catalog v5. Operational telemetry and historical incidents are author-provided observations and are labeled as such.